Reviewer Pack — Minimal Rank of Algebraic K-Theory over Boolean Functions vs…

Entry be49a7bf101a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 07:15:58 UTC

Minimal Rank of Algebraic K-Theory over Boolean Functions vs SOS Degree for Max-CUT

Entry ID: `be49a7bf101a` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-25 07:15:58 UTC

1. Conjecture

Field A (mathematical branch): Algebraic K-Theory **Field B** (complexity object): Sum-of-Squares Complexity: SOS Degree for Max-CUT

Statement:

> {'s1': 'For every degree-d Sum-of-Squares (SOS) approximation α to the Max-CUT problem, there exists a Boolean function f with at most 2n variables such that the minimal rank of its associated K-theory group is at least d log^2 n.', 's2': 'Furthermore, for any fixed k, there exist instances where this inequality holds even when the SOS degree is k.', 's3': 'The minimal rank of the K-theory group for a Boolean function f is defined as the minimum number of generators required to express all elements of the K-group associated with f.'}

Rationale (proposer's reasoning):

> {'s1': 'Algebraic K-Theory provides a rich framework for studying algebraic properties of spaces, which may offer new insights into the structure of Boolean functions.', 's2': 'By connecting this theory to SOS complexity, we aim to uncover deeper connections between algebraic structures and computational problems.', 's3': 'This conjecture, if true, would provide a novel approach to understanding the limitations of SOS-based approximation algorithms for Max-CUT.'}

Taxonomy category: `SOS_DEGREE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `2535816b6818ea37`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The experiment supports the conjecture if, for at least 80% of the generated Boolean functions with up to 2n variables, the minimal rank of the K-theory group is at least d log^2 n, where d is the SOS degree and n is the number of variables. The criterion is falsified if any seed produces a minimal rank less than d log^2 n for more than 20% of the functions or if any seed results in a minimal rank greater than d log^2 n.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.4s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def k_theory_rank(f):
        # Placeholder implementation of K-theory rank calculation
        # This is a dummy function and should be replaced with actual logic
        return len(f)

    def sos_approximation(n, d):
        # Placeholder implementation of SOS approximation
        # This is a dummy function and should be replaced with actual logic
        return random.randint(1, d)

    n = 40
    d = sos_approximation(n, 5)  # Example SOS degree
    f = generate_boolean_function(n)
    rank = k_theory_rank(f)

    metric_value = rank
    conjecture_holds = rank >= d * math.log2(n) ** 2
    counterexample = "" if conjecture_holds else "SOS degree too low"

    return {
        "metric_name": "K-theory rank",
        "metric_value": metric_value,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
```

```
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 31)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results) and sum(1 for r in results if not r["conj
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample='SOS degree too low' first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The experiment did not complete due to a timeout, which prevents us from verifying whether the pre-registered support condition of at least 80% of generated Boolean functions meet the required minimal rank. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	10488
2	propose	ollama_remote	glm4:latest	0	0	11165

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	preregistration	ollama_remote	glm4:latest	0	0	6216
4	novelty	ollama_remote	glm4:latest	0	0	4904
5	novelty	ollama_remote	glm4:latest	0	0	4981
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11402
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7550
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9121
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7151
10	judge	ollama_remote	glm4:latest	0	0	12427

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 85405 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/be49a7bf101a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/be49a7bf101a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/be49a7bf101a.tar.gz` (if generated)